POLITECNICO DI TORINO

Corso di Laurea in Cybersecurity

Tesi di Laurea

An incremental approach to automatic firewall configuration

Relatori
prof. Fulvio Valenza
prof. Daniele Bringhenti
prof. Francesco Pizzato

Candidato
Filippo Del Ministro

Anno accademico 2025-2026

Modern enterprise networks are complex, distributed systems in which security policy enforcement is a critical and continuous operational challenge. As topologies grow in scale and heterogeneity, the manual configuration of packet-filtering firewalls becomes increasingly error-prone. Misconfigurations — whether overly permissive or overly restrictive — can expose sensitive resources or degrade service availability. The shift toward microsegmentation, distributing enforcement across multiple firewall instances throughout the network, improves security granularity but amplifies the management burden. Automated tools that generate formally correct and optimal configurations are therefore essential.

VEREFOO (VErification REasoning on Firewall Optimization) addresses this need by automatically computing firewall allocation and configuration from a network topology and a set of Network Security Requirements (NSRs). It encodes the problem as a Maximum Satisfiability Modulo Theories (MaxSMT) instance solved by the Z3 solver, guaranteeing formal correctness and minimality of the result. However, VEREFOO follows a monolithic approach: every time the NSR set changes, the entire problem is re-encoded and solved from scratch, scaling poorly as the network or requirement set grows.

React-VEREFOO addresses this by taking an incremental approach. It identifies which requirements are new, removed, or unchanged, and restricts the solver to only the affected parts of the network. This reduces reconfiguration cost when few requirements change, but the benefit diminishes when a large fraction of the NSR set is modified simultaneously.

This thesis pushes the incremental principle to its extreme. The proposed Iterative React-VEREFOO introduces new NSRs one at a time: at each step, the current configuration is used as the starting state and exactly one requirement is added. React-VEREFOO computes the minimal update, and the result becomes the input to the next iteration. By construction, the reconsidered portion of the network is always minimal, keeping each MaxSMT subproblem small regardless of the total number of requirements.

The approach has been implemented as a modification of the React-VEREFOO codebase and evaluated on three benchmark topologies: the Stanford university network, the CESNET academic network, and the Texas 2000 synthetic topology. Results show that the iterative method scales linearly with the number of NSRs, maintaining consistent execution times as the requirement set grows. On configurations where vanilla VEREFOO encounters unsatisfiable instances or prohibitive runtimes, the iterative method continues to produce valid results, confirming the viability of fine-grained incremental reconfiguration for network security automation in dynamic environments.

Index

Capitolo 1

Introduction

Capitolo 2

Background

2.1 Context

Network security is a critical concern for virtually every organization, from small businesses to large enterprises and public institutions. All of them rely on computer networks to support their operations, and all of them are exposed to the risk of unauthorized access, data breaches, and service disruptions. As a result, enforcing a well-defined and correct security policy is not optional, but instead is a fundamental operational requirement.

Modern enterprise networks are characterized by growing complexity: hundreds of interconnected nodes, heterogeneous services, and security policies that must be consistently enforced across distributed infrastructure. Moreover, network topologies are not static: new services are deployed, nodes are added or removed, and connectivity requirements evolve continuously, making security management an ongoing and demanding task. The cost of managing network security in this context is significant, as it requires specialized expertise, advanced planning, and constant monitoring.

A central challenge in this landscape stems from the evolution of network security architectures. Traditionally, a single perimeter firewall was sufficient to protect a network by controlling traffic at its boundary. However, as networks grew in scale and complexity, this model proved inadequate: a single point of control cannot enforce fine-grained policies across a heterogeneous internal topology. Modern networks have therefore evolved toward microsegmentation, deploying multiple packet-filtering firewall instances strategically distributed across the topology to enforce security policies at a much more granular level. This distributed approach offers clear advantages such as enabling finer-grained traffic control, reduces single points of failure, and allows security policies to be enforced closer to the traffic sources, but it also introduces considerable management complexity. Each firewall instance must be individually configured with a set of filtering rules, and those rules must collectively enforce the intended security policy across the entire network, without conflicts or gaps.

In this context, the configuration of firewalls becomes a critical task. On one hand, overly permissive rules may allow unauthorized traffic to reach sensitive resources, exposing the network to attacks. On the other hand, overly restrictive rules may block legitimate traffic, degrading service availability and causing operational disruptions. Both failure modes carry significant consequences, and yet both are common outcomes of manual configuration process which is inherently prone to errors especially for large-scale networks.

The gap between the complexity of modern networks and the limitations of manual security management calls for automated approaches that can generate correct and optimal firewall configurations without human intervention. It is in this context that VEREFOO was developed: a tool for the automated, formally correct allocation and configuration of distributed firewalls, designed to bridge exactly this gap.

2.2 VEREFOO

2.2.1 Overview

VEREFOO (VErification REasoning on Firewall Optimization) is a tool designed to automate the allocation and configuration of packet-filtering firewalls in a network, eliminating the need for manual intervention entirely. Given a network topology and a set of Network Security Requirements (NSRs) — high-level specifications describing which traffic flows must be allowed or denied between network endpoints — VEREFOO automatically determines where firewalls should be placed within the topology and generates the corresponding filtering rules for each allocated instance. The resulting configuration is expressed in a vendor-independent format, which can then be translated into concrete rules for specific implementations such as iptables, BPF, or OVS.

The tool provides three formal guarantees. *Automation* ensures that no human intervention is required at any stage of the process. *Formal correctness* guarantees that the generated configuration satisfies all specified NSRs by construction, eliminating the risk of undetected misconfigurations. *Optimality* ensures that both the number of allocated firewall instances and the number of generated rules are minimised, reducing resource consumption and improving performance. These properties are achieved by encoding the firewall allocation and configuration problem as a partial weighted Maximum Satisfiability Modulo Theories (MaxSMT) instance, which is then solved using the Z3 SMT solver. The hard constraints of the problem encode the correctness requirements, while the soft constraints encode the optimality objectives; the solver finds a solution that satisfies all hard constraints and maximises the sum of satisfied soft constraint weights.

- todo: Spiegare il concetto di Service Graph - todo: Spiegare il concetto di Firewall Allocation Schema (FAS) => immagine FAS - check: Verefoo non ragiona su singoli pacchetti, ma su flussi di traffico => spiegare benefici

VEREFOO has been validated on both synthetic and real-world network topologies, including well-known benchmarks such as the Stanford and CESNET networks, demonstrating reliability and scalability across a range of network sizes and requirement sets [**?**].

Despite these strengths, VEREFOO's monolithic approach has a fundamental scalability limitation: the entire set of NSRs and the full network topology are encoded into a single MaxSMT problem. As the number of requirements or the size of the network grows, the complexity of the problem increases significantly, leading to prohibitive computation times. This limitation motivates the development of more efficient alternatives, starting with React-VEREFOO.

2.2.2 React-VEREFOO

In operational environments, network security policies are not static: NSRs evolve over time as new services are deployed, topology changes occur, or threat models are updated. A fundamental limitation of the monolithic approach used by VEREFOO is that any change to the NSR set triggers a full recomputation of the firewall configuration from scratch. React-VEREFOO addresses this by offering a more efficient approach in the dynamic context described above, avoiding the need to discard a valid existing configuration when only a small portion of the requirements has changed.

React-VEREFOO takes as input both the existing firewall configuration and the updated set of security requirements, and computes only the minimal changes needed to bring the network into compliance. Rather than reconsidering the entire problem, it identifies which requirements are new, which have been removed, and which remain unchanged. Only the newly added requirements drive the reconfiguration: the approach determines which parts of the network are affected and restricts the solver to those areas, leaving the rest of the configuration untouched. The result is an updated firewall allocation and rule set that satisfies all current requirements, with the same guarantees of correctness and optimality as VEREFOO.

Experimental results show that React-VEREFOO is significantly more efficient than vanilla VEREFOO when the proportion of modified NSRs is small, as the reduced problem size leads

to much shorter solving times. However, when a large fraction of the requirements changes, the benefits is reduced and React-VEREFOO's performance becomes comparable to that of the monolithic approach. This observation motivates the approach investigated in this thesis: by pushing the incremental logic to its extreme - introducing only one new NSR per iteration - the proportion of modified requirements is always minimised, potentially achieving consistent speedups regardless of the total number of requirements.

Capitolo 3

Thesis Objectives

3.1 Idea of the Solution

- estremizzare l'approccio di React Verefoo: se React Verefoo é più veloce di Verefoo quando il numero di NSR aggiuntivi é piccolo, allora proviamo ad introdurre un solo NSR alla volta. In questo modo il numero di NSR modificati ad ogni iterazione é sempre minimo, e quindi l'overhead introdotto dall'approccio iterativo dovrebbe essere sempre compensato dal fatto che il problema da risolvere ad ogni iterazione é molto più piccolo.

As discussed in the introduction, React-VEREFOO is most effective when the number of additional requirements is small relative to the total. The key idea of the approach is to push that observation to its logical extreme: instead of processing all new NSRs at once, the proposed approach introduces them one at a time, turning the configuration problem into a sequence of minimal reconfiguration steps.

The process works as follows. At each iteration, **the current firewall configuration is treated as the initial state and one single new NSR is added as the target**. React-VEREFOO is then invoked to compute the minimal update needed to satisfy that one additional requirement. The output of each iteration, meaning the updated allocation scheme and the associated filtering rules becomes the initial state for the next one. This continues until all NSRs have been incorporated.
Formally, if the set of additional NSRs to be processed is $\{A, B, C, \ldots\}$ and P_0 denotes the initial set of properties already enforced in the network, the sequence of iterations is:

1. initial: P_0, additional: $\{A\}$
2. initial: $\{P_0, A\}$, additional: $\{B\}$
3. initial: $\{P_0, A, B\}$, additional: $\{C\}$
4. …

Since each invocation of React-VEREFOO deals with exactly one added requirement, the portion of the network that must be reconsidered at each step is minimised by construction. This keeps each individual MaxSMT subproblem small, regardless of the total number of NSRs, with the expectation of reducing overall computation time. The number of iteration steps is equal to the number of new NSRs, which is a linear factor, but the reduced complexity of each step is expected to more than compensate for that.

Capitolo 4

The Iterative VEREFOO Approach

Capitolo 5

Implementation and Validation

5.1 Implementation

- serializer -

5.2 Testing

5.2.1 Test Presentation

5.2.2 Test Configurations

5.2.3 Results

Figure 5.1 reports the average execution times measured across the three benchmark topologies. Each data point represents the mean over 20 runs. The Y-axis uses a logarithmic scale to accommodate the wide range of values observed, particularly for larger networks.

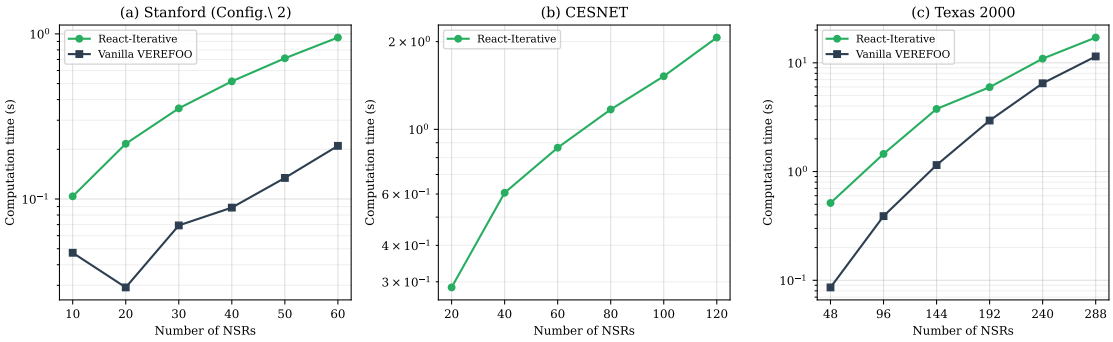

Figura 5.1. Execution time comparison between React-Iterative VEREFOO and Vanilla VEREFOO across three benchmark topologies.

5.2.4 Comparison and Discussion

Capitolo 6

Conclusion and Future Work